THE NARRATIVE JOURNEY

The Story of Turning a Wild Spreadsheet into Knowledge

How GW2-UME transforms raw community notes into mathematically verified graph triples

THE SCENE: A REAL PLAYER'S NOTEBOOK

ðŸ“• File Grounding:

`data/sample_tables/google_sheet_hope_bifrost_tracker.csv`

â€¢ The Context:

A player created a Google Sheet to track crafting the legendary pistol 'HOPE' and its precursor 'Prototype'.

â€¢ The Clutter:

- No clean single table: 4 matrices pasted side-by-side.
- Cells contain messy pairs: 'Crystalline Ingot, 250', 'Prototype, 1', 'Gift of Condensed Magic, 2'.
- No foreign keys or database relationships.

THE MISSION: ZERO GUESSWORK STRUCTURE

â€¢ What We Want to Achieve:

1. Automatically untangle the messy columns.
2. Separate numbers from item names without errors.
3. Prove what each column means using the game rulebook.
4. Emit a permanent, connected Knowledge Graph.

â€¢ The 5 Acts of the Story:

- â€¢ Act 1: The Detective (Text Normalization)
- â€¢ Act 2: The Memory Bank (Dense Vector Search)
- â€¢ Act 3: The Family Tree (Least Common Subsumer)
- â€¢ Act 4: The Jigsaw Mesh (Constraint Solver)
- â€¢ Act 5: The Handshake (Neuro-Symbolic Convergence)

ACT 1: THE DETECTIVE (CLEANING & STRUCTURING)

How raw strings like 'Crystalline Ingot, 250' become structured semantic units

WHERE THIS CODE LIVES

ðŸ“• Code Files:

- â€¢ src/gw2_ume/normalization/text_cleaner.py
- â€¢ src/gw2_ume/normalization/llm_normalizer.py

â€¢ What the Detective Does:

1. Looks at messy cell values.
2. Uses regex patterns to split numbers from names:

   ```
   'Crystalline Ingot, 250' -> ('Crystalline Ingot', 250)
   ```

3. Translates game slang into official game names:

   ```
   'clovers' -> 'Mystic Clover'
   'amalgams' -> 'Amalgamated Gemstone'
   ```

4. Packages everything into a clean TableGrid matrix.

BEFORE & AFTER TRANSFORMATION

Raw Input Cell String:

```
"Crystalline Ingot,250"
```

Structured EntitySpan Object:

```
EntitySpan(
  text='Crystalline Ingot',
  normalized_text='Crystalline Ingot',
  candidate_types=['CraftingMaterial'],
  quantity=250
)
```

Why this matters in plain English:

The number '250' is no longer just meaningless text; it is now a recognized numerical quantity permanently attached to the Crystalline Ingot material.

ACT 2: THE MEMORY BANK (VECTOR RECOGNITION)

How the AI matches words to concepts in milliseconds using Apple Silicon MPS acceleration

WHERE THIS CODE LIVES

ðŸ“• Code Files:

- `â€¢ src/gw2_ume/indexing/embedder.py`
- `â€¢ src/gw2_ume/indexing/faiss_index.py`

â€¢ What the Memory Bank Does:

1. Takes the cleaned words ('HOPE', 'Prototype').
2. Runs them through all-MiniLM-L6-v2 to make a 384-dimensional mathematical fingerprint.
3. Compares the fingerprint against all known game items stored in the FAISS vector index.
4. **Runs on Apple Silicon GPU (MPS) in ~4 milliseconds.**

WHAT THE MEMORY BANK RETRIEVES

Word: 'HOPE'
`-> gw2leg:HOPE`
Type: LegendaryWeapon (99.2% match)

Word: 'Prototype'
`-> gw2leg:Prototype`
Type: PrecursorWeapon (98.5% match)

Word: 'Gift of Condensed Magic'
`-> gw2:GiftOfCondensedMagic`
Type: ComponentItem (99.4% match)

Word: 'Crystalline Ingot'
`-> gw2:CrystallineIngot`
Type: CraftingMaterial (97.8% match)

The Problem Still Remaining:

We know what each word means in isolation, but how do they fit together? What is the whole column about? That requires the Ontology Rulebook (Act 3).

ACT 3: THE FAMILY TREE (DEDUCING COLUMN MEANING)

How the Least Common Subsumer (LCS) mathematically proves what a mystery column represents

WHERE THIS CODE LIVES

ðŸ“• Code & Ontology Files:

- â€¢ ontologies/gw2_core.ttl
- â€¢ src/gw2_ume/ontology/reasoner.py
- â€¢ src/gw2_ume/matching/cta.py

â€¢ The Mystery Column:

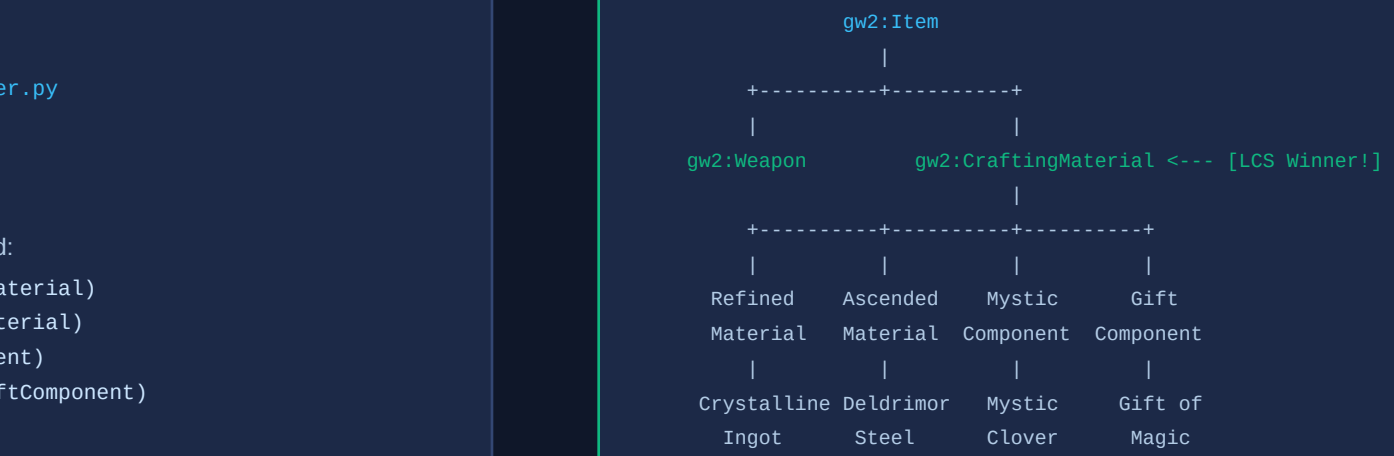
Column 0 had no header, but contained:

- â€¢ Crystalline Ingot (RefinedMaterial)
- â€¢ Deldrimor Steel (AscendedMaterial)
- â€¢ Mystic Clover (MysticComponent)
- â€¢ Gift of Condensed Magic (GiftComponent)

â€¢ The Question:

What single class encompasses all these items?

THE ONTOLOGY FAMILY TREE



The Mathematical Verdict:

LCS proves Column 0 is 'gw2:CraftingMaterial' (90% conf).

It avoids guessing overly generic 'Item' or wrong 'Weapon'.

ACT 4 & 5: THE JIGSAW MESH & THE HANDSHAKE

Connecting rows into relations and running the Neuro-Symbolic sanity check

ACT 4: THE JIGSAW PUZZLE (MESH SOLVER)

ðŸ“• Code Files:

- `â€¢ src/gw2_ume/matching/mesh_solver.py`
- `â€¢ src/gw2_ume/matching/cpa.py`

â€¢ Connecting the Columns:

- Col 0 is CraftingMaterial.
- Col 1 is Quantity (e.g. 250).
- The target recipe is HOPEMysticForgeRecipe.

â€¢ Applying the Property Constraint:

gw2:requiresMaterial has:

`Domain = CraftingRecipe, Range = CraftingMaterial.`

Both sides match 100%! The puzzle locks together:

`<HOPE_Recipe> requiresMaterial <Crystalline Ingot>`
`with quantity = 250.`

ACT 5: THE HANDSHAKE (PING-PONG CHECK)

ðŸ“• Code Files:

- `â€¢ src/gw2_ume/pipeline/pingpong.py`
- `â€¢ src/gw2_ume/pipeline/engine.py`

â€¢ The Dialogue:

1. Neural AI proposes:
   'Here is my interpretation of HOPE's 32 ingredients.'
2. Symbolic Reasoner evaluates:
   'Checking all 32 ingredients against OWL 2 axioms...'
   - `Zero domain/range violations.`
   - `Quantities match positive integers.`
   - `Precursor 'Prototype' verified in slot 1.`

â€¢ Result: Converged in 1 pass, 100% confidence.

EPILOGUE: THE LIVING KNOWLEDGE GRAPH

The end result: Clean W3C RDF Turtle triples and an interactive visual dashboard

ðŸ“• Output Artifacts Generated by pipeline/enricher.py & ui/visualizer.py:

```
# 1. The Verified HOPE Legendary Recipe in Turtle Syntax
gw2leg:HOPE a gw2:LegendaryWeapon ;
    rdfs:label "HOPE" ;
    gw2:hasPrecursor gw2leg:Prototype ;
    gw2:craftedWithRecipe gw2leg:HOPEMysticForgeRecipe .

gw2leg:HOPEMysticForgeRecipe a gw2:MysticForgeRecipe ;
    gw2:hasMysticForgeIngredient gw2leg:Prototype ,
                                 gw2leg:GiftOfHOPE ,
                                 gw2leg:MysticTribute ,
                                 gw2leg:GiftOfMaguumaMastery ;
    gw2:producesItem gw2leg:HOPE .

# 2. The Granular Ingredient Triple with Quantity Binding
gw2leg:HOPEMysticForgeRecipe gw2:requiresIngredient gw2item:Crystalline_Ingot ;
                             gw2:hasIngredientQuantity 250 .
```

Interactive Dashboard: Open 'dashboard.html' in your browser to explore the force graph.